

# First, reproduce learning in a simpler setting.

A recursive language model (RLM) can use code and ask helper models to solve parts of a problem. We eventually want to train those choices.

## Today's question

Can we use a published training recipe to improve a model's math answers on questions excluded from training?

We use the authors' code for **Dr. GRPO**, a reinforcement learning (RL) method. It learns from scores assigned to the model's own answers.

This is a small reproduction of an existing method. It establishes a reference for future work on recursive models.

Liu et al., *Understanding R1-Zero-Like Training: A Critical Perspective* (2025).

Official code; our run is much smaller than the paper's experiments.

# Reward correct attempts, then change the model.

**Illustrative question:** Solve  $3x + 2 = 14$ . The correct answer is  $x = 4$ .

| The model's attempts            | Checker score | Learning direction      |
|---------------------------------|---------------|-------------------------|
| Two answers end in $x = 4$ .    | 1 each        | Make these more likely. |
| Two end in $x = 5$ or $x = 6$ . | 0 each        | Make these less likely. |

Each attempt is compared with the average score for that question. These differences guide changes to the model's numerical settings, or **weights**.

The checker judges the final answer. We do not supply worked solutions for the model to copy.

Simplified illustration with four attempts; our runs use eight. If every attempt receives the same score, this question supplies no relative learning signal.

# We train on math, then test on different questions.

Our short runs used **Qwen2.5-Math-1.5B**, the same 512 math questions, eight attempts per question, and 32 weight updates on one A100 GPU.

We tested the starting and trained models on the same **500 MATH500 questions**, all excluded from weight training.

We also tested two ways of asking each question.

**Chat style:** Put the question inside a conversation, with an instruction to explain the steps and mark the final answer.

**Question alone:** Give only the original question.

These three runs started from the original model: one used chat style and two used questions alone.

Of the 500 questions, 64 were used to monitor progress. We did not choose the best checkpoint. Earlier pretraining exposure is unknown.

# The input format changes the size of the gain.

Correct answers out of the same 500 MATH500 questions.

| Model tested                    | Chat-style<br>input | Question<br>alone |
|---------------------------------|---------------------|-------------------|
| Starting model                  | 154                 | 305               |
| Trained with chat style         | 308                 | 317               |
| Trained with questions alone    | 168                 | 314               |
| Repeated question-only training | 166                 | 306               |

Question-only training scored 314 and 306, versus 305 before training. We have not established a reliable gain over that stronger starting setup.

Each trained row is a separate run at its final checkpoint. The same checker and length limit apply throughout. Scores judge final answers, not every reasoning step.

# A higher score does not always mean a new skill.

**An actual test question:** Write  $3/20$  as a decimal.

The saved responses, summarized

**Before:** The model repeated the question until it hit the length limit.

**After:** It explained the division and answered 0.15.

**Separate reference check:** The authors' released model scored 366/500 (73.2%), versus 74.2% reported. We evaluated their model; we did not train it.

**Now running:** A larger recipe from the original model: roughly 4,000 questions and about 250 planned updates. Results are pending.

The example is from our earlier 128-question chat-style test. Testing released weights does not reproduce their training or establish better RLM decomposition.